

HW6 Solutions

Since I didn't make this problem, these solutions may not be the most efficient.

In the following parts, notate the label of a vertex v as $\ell(v)$ and the set of its children C_v . Also notate the subtree rooted at vertex v as $T_v = (V_v, E_v)$.

Problem 1a Solution

Design

First, run DFS to obtain a set of all distinct labels L . Now in words, we define

- $\text{LRMB}_L[v][l]$ as the **size of the largest boring rooted minor of T_v such that all of v 's children have label l** .
- $\text{LRMB}[v]$ as the **size of the largest boring rooted minor of T_v** . By definition, $\text{LRMB}[v] = \max_{l \in L} \text{LRMB}_L[v][l]$.

We perform a post-order traversal of T , armed with the following recursive formula:

$$\text{LRMB}_L[v, l] = 1 + \sum_{c \in C_v} \begin{cases} \text{LRMB}[c] & \text{if } \ell(c) = l \\ \text{LRMB}_L[c][l] - 1 & \text{otherwise.} \end{cases}$$

Correctness

A valid explanation is as follows: The term "1" at the beginning of the formula accounts for the vertex v itself. Next, we go through each child c of the vertex v and accumulate the sizes of the largest boring rooted minors rooted at these children into the sum. There are two cases to consider:

1. $\ell(c) = l$: If the label $\ell(c)$ of a child c matches the target label l , that means this subtree rooted at c satisfies our 'boring' condition for this case. We can safely include the entire subtree rooted at c in our rooted minor. Hence, we add $\text{LRMB}[c]$ to our sum.
2. $\ell(c) \neq l$: If the label $\ell(c)$ of the child c does not match the target label l , then c itself cannot be included in this boring rooted minor (as it would break the 'boring' requirement for v). However, we still have a chance to include some or all of the boring rooted minor that is rooted at c and satisfies the label l among its own children. That's what $\text{LRMB}_L[c][l]$ is. Since we can't include c itself, we have to subtract 1: thus, we add $\text{LRMB}_L[c][l] - 1$ to our sum.

We use post-order traversal to ensure that all the necessary values for the children are available when calculating the value for the parent. Our final answer is given by $\text{LRMB}[v_{\text{root}}]$.

Runtime

The time complexity of filling up $\text{LRMB}[v]$ is $\mathcal{O}(|V| \times |L|)$, since we have a $|V|$ table whose transitions take $|L|$ time.

Now, for $\text{LRMB}_L[v][l]$, we have a $|V| \times |L|$ table. However, each cell in this table doesn't just take constant time to fill, as it appears to depend on the outdegree for each vertex v due to the sum over C_v in the recursive formula. At first glance, this may suggest a time complexity of $\mathcal{O}(|V| \times |L| \times D)$, where D is the outdegree of the graph.

However, for any graph, the sum of the outdegrees for all vertices is $2(|V| - 1)$, which is $\mathcal{O}(|V|)^1$. Therefore, the time spent on summing over all children for all vertices is effectively $\mathcal{O}(|V|)$, when amortized over the entire table. Thus, the overall time complexity for $\text{LRMB}_L[v][l]$ is actually $\mathcal{O}(|V| \times |L|)$.

Consequently, the overall time complexity for the entire algorithm is $\mathcal{O}(|V| \times |L|)$.

¹known as the *handshaking lemma*.

Problem 1b Solution

Design First, run DFS to obtain a set of all distinct labels L . Now in words, we define

- $\text{LRMHO}_L[v][l]$ as the **size of the largest heap-ordered rooted minor of T_v such that all of v 's children have label less than l** .
- $\text{LRMHO}[v]$ as the **size of the largest heap-ordered rooted minor of T_v** . By definition, $\text{LRMHO}[v] = \max_{l \in L} \text{LRMHO}_L[v][l]$.

We perform a post-order traversal of T , armed with the following recursive formula:

$$\text{LRMHO}_L[v, l] = 1 + \sum_{c \in C_v} \begin{cases} \text{LRMHO}[c] & \text{if } \ell(c) < l \\ \text{LRMHO}_L[c][l] - 1 & \text{otherwise.} \end{cases}$$

Correctness

A valid explanation is as follows: The term "1" at the beginning of the formula accounts for the vertex v itself. Next, we go through each child c of the vertex v and accumulate the sizes of the largest heap-ordered rooted minors rooted at these children into the sum. There are two cases to consider:

Case 1. If the label $\ell(c)$ of a child c is less than the target label l , that means this subtree rooted at c satisfies our 'heap-ordered' condition for this case. We can safely include the entire subtree rooted at c in our rooted minor. Hence, we add $\text{LRMHO}[c]$ to our sum.

Case 2. If the label $\ell(c)$ of the child c does not match the target label l , then c itself cannot be included in this heap-ordered rooted minor (as it would break the 'heap-ordered' requirement for v). However, we still have a chance to include some or all of the heap-ordered rooted minor that is rooted at c and satisfies the heap-ordering among its own children. That's what $\text{LRMHO}_L[c][l]$ is. Since we can't include c itself, we have to subtract 1: thus, we add $\text{LRMHO}_L[c][l] - 1$ to our sum.

We use post-order traversal to ensure that all the necessary values for the children are available when calculating the value for the parent. Our final answer is given by $\text{LRMHO}[v_{\text{root}}]$.

Runtime

The time complexity of filling up $\text{LRMHO}[v]$ is $\mathcal{O}(|V| \times |L|)$, since we have a $|V|$ table whose transitions take $|L|$ time.

As in part (a), we claim that the time complexity of filling up $\text{LRMHO}_L[v][l]$ is $\mathcal{O}(|V| \times |L|)$. This is because we have a $|V| \times |L|$ table, and each *column* of this table takes time $\mathcal{O}(|V|)$ time to fill.

Consequently, the overall time complexity for the entire algorithm is $\mathcal{O}(|V| \times |L|)$.

Problem 1c Solution

Design In words, we define

- $\text{ISGOOD}[v][p]$ as a boolean value representing for each node, given that these parents are excluded, is this node good? Meaning is it within this range
- $\text{LRMBSO}[v][L]$ as the **number of nodes in T_v that will be present the largest binary-search-ordered rooted minor of T .**

We first build $\text{GOODRANGE}[v]$. For every vertex, store a hashmap of all the parents of v .

We perform a post-order traversal of T , armed with the following recursive formula:

$$\begin{aligned} \text{LRMBSO}[v] &= 1 + \max\{\text{LRMBSO}[c] : c \in V_{v_l} \text{ and } \ell(c) < \ell(v)\} \\ &\quad + \max\{\text{LRMBSO}[c] : c \in V_{v_r} \text{ and } \ell(c) > \ell(v)\}, \\ \text{LRMBSO}[v] &= 1 + \sum_{c \in C_v} \begin{cases} \text{LRMBSO}[c] & \text{if } \text{ISGOOD}[v] \\ \text{LRMBSO}_L[c][L] - 1 & \text{otherwise.} \end{cases} \end{aligned}$$

Correctness

Runtime Consequently, the overall time complexity for the entire algorithm is $\mathcal{O}(|V| \times |L|)$.

Problem 1d Solution

Modification of part (c). I suspect depending on the approach they used in part (c), this will either be very hard for them or very easy.

Problem 1e Solution

Here we discuss the solution to part (e).

So I think many students used DP within DP, which is basically what this is. But, hopefully this solution is a bit more elegant and displays sort of an *intuition* about how a student may come to this solution.

First Pass Solution (Lots of students had something like this in office hours)

So a student may start out with saying ok, let's define

- $\text{LCS}(u, v)$ as **the largest common rooted minor between the trees rooted at u and v .**

And then a student may be like ok, let's think about this as Longest Common Subsequence, and then after a bit of thinking they develop the recursive case

$$\text{LCS}(u, v) = \begin{cases} \max \left\{ \max_{c \in C_u} \text{LCS}(u, c), \max_{c \in C_v} \text{LCS}(c, v) \right\} & \text{if } \ell(u) \neq \ell(v) \\ 1 + ??? & \text{if } \ell(u) = \ell(v) \end{cases}$$

Uh oh, we are stuck! If they match, students could not figure out what to do! Many students think that it can only be done in exponential time. Most students that have submitted solved another DP problem and substituted it in for $???$, which is definitely a correct approach. But now, let's maybe think about doing it in just one DP problem instead, by thinking of trees as *balanced sequences*.

Refined Solution

Disclaimer: here I took the term *balanced sequence* from many (similar but not identical) research papers, but the rest of the idea is mine. I think the term comes from balanced parentheses.

My motivation for this solution came up with a way to translate trees into lists. Thanks to Rahi for unordered inspiration.

We need some way to convert a rooted labeled tree into a rooted labeled sequence. Once we do this.

- (a) Let $\text{LCSB}(s, t)$ represent the longest common rooted tree sequences between two rooted tree sequences s and t . Note that every balanced sequence has a corresponding label, which corresponds to the label of the root.

Suppose we have a sequence s like 0010101100010111. First define

$\text{FCS}(s)$ as everything that's blue. I'm defining $\text{OCS}(s)$ as everything that's red.

Our recursive case is

$$\text{LCSB}(s, t) = \begin{cases} \max \left\{ \text{LCSB}(u, c), \text{LCSB}(c, v) \right\} & \text{if } \ell(u) \neq \ell(v) \\ 1 + \text{LCSB}(\text{FCS}(s), \text{FCS}(t)) + \text{LCSB}(\text{OCS}(s), \text{OCS}(t)) & \text{if } \ell(s) = \ell(t) \end{cases}$$

- (b) Return $\text{LCSB}[\text{root}_1, \text{root}_2]$

- (c) We have a $n_1^2 \times n_2^2$ dimensional table whose transitions take $\mathcal{O}(1)$ time. So our complexity is $\mathcal{O}(n_1^2 \times n_2^2)$.² For labeled case, replace zeroes with labels.

²This can be reduced more, cause we are not really filling up all $n_1^2 \times n_2^2$, but this is good enough